

```
package jp.co.active.kenshu.diary;

import java.time.DateTimeException;
import java.time.LocalDate;
import java.util.HashMap;
import java.util.Map;

/**
 * 日記入力に関するバリデーションを行うクラス
 */
public class DiaryValidator {

    /**
     * 日記入力画面を表示するときの年月日チェックを行う
     *
     * @param year 年
     * @param month 月
     * @param day 日
     * @return バリデーション結果
     */
    public Result validateDate(String year, String month, String day) {

        Result result = new Result();

        try {
            // null または空文字チェック
            if (isBlank(year) || isBlank(month) || isBlank(day)) {
                result.addError("date", "年月日が正しくありません。");
                return result;
            }

            // 数字以外チェック
            if (!isNumber(year) || !isNumber(month) || !isNumber(day)) {
                result.addError("date", "年月日は数字で指定してください。");
                return result;
            }

            int yearNumber = Integer.parseInt(year);
            int monthNumber = Integer.parseInt(month);
            int dayNumber = Integer.parseInt(day);
```

```

// 存在しない年月日の場合、ここで DateTimeException が発生する
LocalDate targetDate = LocalDate.of(yearNumber, monthNumber, dayNumber);

// 未来日チェック
if (targetDate.isAfter(LocalDate.now())) {
    result.addError("date", "未来の日付の日記は入力できません。");
    return result;
}

result.setTargetDate(targetDate);

} catch (NumberFormatException e) {
    result.addError("date", "年月日は数字で指定してください。");

} catch (DateTimeException e) {
    result.addError("date", "存在しない年月日です。");

} catch (Exception e) {
    result.addError("date", "年月日の確認中にエラーが発生しました。");
}

return result;
}

/**
 * 確認画面に進む前の日記入力チェックを行う
 *
 * @param year 年
 * @param month 月
 * @param day 日
 * @param title タイトル
 * @param main 本文
 * @param feel 今日の気分
 * @param weather 今日の天気
 * @return バリデーション結果
 */
public Result validateDiary(
    String year,
    String month,
    String day,

```

```
String title,
String main,
String feel,
String[] weather) {

Result result = validateDate(year, month, day);

// タイトル未入力チェック
if (isBlank(title)) {
    result.addError("title", "*必須入力です。");
}

// 本文未入力チェック
if (isBlank(main)) {
    result.addError("main", "*必須入力です。");
}

// 気分チェック
if (isBlank(feel)) {
    result.addError("feel", "*必須選択です。");

} else if (!isValidFeel(feel)) {
    result.addError("feel", "不正な値が送信されました。");
}

// 天気チェック
if (weather == null || weather.length == 0) {
    result.addError("weather", "*必須選択です。");

} else if (!isValidWeather(weather)) {
    result.addError("weather", "不正な値が送信されました。");
}

return result;
}

/**
 * XSS 対策用に HTML 特殊文字をエスケープする
 *
 * @param value 文字列
 * @return エスケープ後の文字列

```

```
*/  
public String escapeHtml(String value) {
```

```
    if (value == null) {  
        return "";  
    }
```

```
    return value  
        .replace("&", "&amp;")  
        .replace("<", "&lt;")  
        .replace(">", "&gt;")  
        .replace("\"", "&quot;")  
        .replace("'", "&#39;");  
}
```

```
/**  
 * 配列の中に指定した値が含まれているか判定する  
 *  
 * @param values 配列  
 * @param target 探す値  
 * @return 含まれていればtrue  
 */
```

```
public boolean contains(String[] values, String target) {
```

```
    if (values == null) {  
        return false;  
    }
```

```
    for (String value : values) {  
        if (target.equals(value)) {  
            return true;  
        }  
    }
```

```
    return false;  
}
```

```
/**  
 * null または空文字か判定する  
 *  
 * @param value 判定する文字列
```

```

* @return null または空文字なら true
*/
private boolean isBlank(String value) {
    return value == null || value.trim().isEmpty();
}

/**
 * 数字だけか判定する
 *
 * @param value 判定する文字列
 * @return 数字だけなら true
 */
private boolean isNumber(String value) {
    return value.matches("[0-9]+");
}

/**
 * 気分の値が想定内か判定する
 *
 * @param feel 今日の気分
 * @return 想定内なら true
 */
private boolean isValidFeel(String feel) {
    return "good".equals(feel)
        || "soso".equals(feel)
        || "bad".equals(feel);
}

/**
 * 天気値が想定内か判定する
 *
 * @param weather 今日の天気
 * @return 想定内なら true
 */
private boolean isValidWeather(String[] weather) {

    for (String value : weather) {
        if (!"晴".equals(value)
            && !"曇".equals(value)
            && !"雨".equals(value)
            && !"雪".equals(value)) {

```

```

        return false;
    }
}

return true;
}

/**
 * バリデーション結果を持つクラス
 */
public static class Result {

    /** エラーメッセージ */
    private Map<String, String> errors = new HashMap<>();

    /** チェック済みの日付 */
    private LocalDate targetDate;

    /**
     * エラーを追加する
     *
     * @param key エラーの種類
     * @param message エラーメッセージ
     */
    public void addError(String key, String message) {
        errors.put(key, message);
    }

    /**
     * エラーがあるか判定する
     *
     * @return エラーがあれば true
     */
    public boolean hasError() {
        return !errors.isEmpty();
    }

    /**
     * エラーメッセージを取得する
     *
     * @return エラーメッセージ

```

```
*/
public Map<String, String> getErrors() {
    return errors;
}

/**
 * チェック済みの日付を取得する
 *
 * @return チェック済みの日付
 */
public LocalDate getTargetDate() {
    return targetDate;
}

/**
 * チェック済みの日付を設定する
 *
 * @param targetDate チェック済みの日付
 */
public void setTargetDate(LocalDate targetDate) {
    this.targetDate = targetDate;
}
}
}
```